

ПРАКТИКУМ

Локальные AI-агенты, Git и DevOps

Codex Edition · собственный минимальный runtime gateway

Windows · PowerShell · Open-LLM-VTuber · Ollama · OpenRouter/OpenAI

Версия 1.0 · август 2026

Главная идея. Codex выполняет инженерную работу в репозитории. Отдельный минимальный gateway даёт приложению постоянный OpenAI-совместимый HTTP endpoint. Эти роли не смешиваются.

Отметки автора / преподавателя: _____

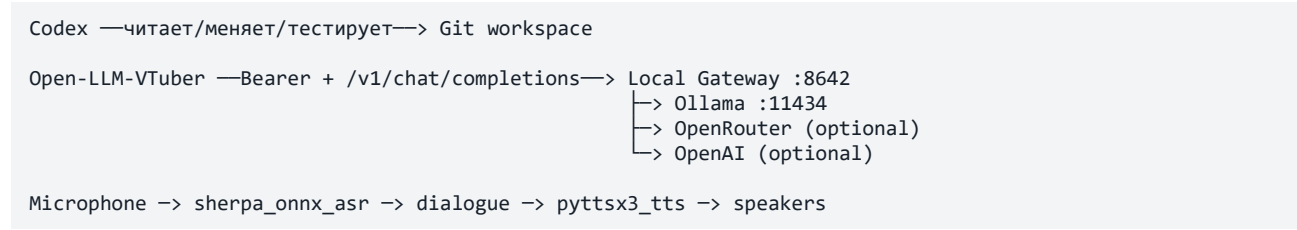
Как пользоваться практикумом

Это самостоятельная редакция практикума для работы с Codex. Она сохраняет учебную траекторию — инвентаризация, baseline, локальный голос, gateway, отказоустойчивость и поставка — но заменяет связку Kimi Code + Hermes Agent на Codex + отдельный ограниченный gateway.

Если уже выполнены эксперименты 1–8. Не начинайте заново. Выполните контрольную точку 8.5, повторите только пункты без доказательств и продолжайте с эксперимента 9.

Компонент	Роль	Не является
Codex	Анализ, план, правки, тесты, Git	Постоянным HTTP runtime gateway
Минимальный gateway	Loorback API, auth, маршрутизация моделей	Агентом разработки
Ollama	Локальный LLM/VLM backend	Облачным API
OpenRouter / OpenAI	Опциональный облачный backend	Локальным offline baseline

Целевая архитектура



Правила безопасности

- Секреты хранятся только в локальном .env, который игнорируется Git; значения не выводятся в логи.
- Gateway слушает только 127.0.0.1, требует Bearer token и имеет allowlist backend/model.
- Cloud backends отключены по умолчанию. Голосовые тесты не отправляют аудио во внешние API.
- Каждое изменение выполняется отдельным атомарным шагом после подтверждения score.

Подготовка рабочего места

Рекомендуемая структура

```
C:\dev\agent-lab\
├── Open-LLM-VTuber\      # fork/upstream exercise repository
├── local-agent-gateway\  # отдельный учебный gateway
└── evidence\             # обезличенные логи и снимки
```

Предварительные требования

Инструмент	Проверка
Git	git --version
GitHub CLI	gh auth status
uv	uv --version
Ollama	ollama --version
Codex Desktop/CLI	открывается workspace C:\dev\agent-lab

Windows PowerShell. Bash heredoc вида `python - <<'PY'` в PowerShell не работает. Используйте `here-string: @' ... '@ | uv run python -.`

```
Set-Location C:\dev\agent-lab
git --version
gh auth status
uv --version
ollama --version
```

Контракт контролируемого агента

Перед каждым изменением вставляйте этот контракт в задачу Codex. Он отделяет планирование от записи и делает доказательства воспроизводимыми.

Работай как инженер в режиме контролируемого агента.

До изменения файлов:

1. Покажи карту релевантной части репозитория и входные точки.
2. Сформулируй одну атомарную цель.
3. Перечисли файлы, которые собираешься читать и менять.
4. Назови риски, предположения и команды проверки.
5. Сначала покажи план. Не меняй файлы до моего явного одобрения.

Ограничения:

- не используй `sudo`, `force push` и опасные команды очистки;
- не переключай и не изменяй `main`;
- не читай и не печатай `.env`, токены, SSH-ключи, `cookies` и `keychain`;
- не трогай файлы за пределами текущего репозитория;
- не ставь глобальные пакеты без отдельного обоснования;
- не скрывай ошибки и не подменяй тест словесным утверждением;
- после изменения покажи `diff`, реальный минимальный тест и `git status --short`;
- если нужен больший `score`, остановись и запроси новый атомарный шаг.

Эксперимент 1. Инвентаризация среды

Зафиксировать исходную среду без изменений.

Атомарная цель. Зафиксировать исходную среду без изменений.

Scope

Читать	Менять
README, quick-start, версии инструментов	Ничего

Порядок выполнения

Откройте C:\dev\agent-lab в Codex и запросите только read-only аудит.

```
Set-Location C:\dev\agent-lab
Get-ChildItem -Force
git --version
gh auth status
uv --version
ollama list
```

Контроль. Не включайте в отчёт token, домашние пути и иные персональные значения.

Артефакты и доказательства

- [] Таблица версий
- [] Список доступных локальных моделей без персональных путей
- [] git status для существующих репозиториев

Критерий завершения

- [] Команды завершились реальным выводом
- [] Ни один файл не изменён

Эксперимент 2. Fork и рабочая ветка

Подготовить безопасную ветку в собственном fork.

Атомарная цель. Подготовить безопасную ветку в собственном fork.

Score

Читать	Менять
Git remotes, branch, status	Только новая ветка в fork

Порядок выполнения

```
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
git remote -v
git branch --show-current
git status --short
```

Если текущая ветка main, создайте отдельную ветку только после подтверждения плана.

```
git switch -c lab/codex-baseline
```

Артефакты и доказательства

- [] origin указывает на fork
- [] upstream указывает на исходный проект
- [] Имя рабочей ветки

Критерий завершения

- [] main не изменён
- [] Рабочее дерево изначально зафиксировано

Эксперимент 3. Карта репозитория с Codex

Получить карту входных точек, конфигурации и тестов без записи.

Атомарная цель. Получить карту входных точек, конфигурации и тестов без записи.

Scope

Читать	Менять
README.md, CLAUDE.md/AGENTS.md, pyproject.toml, quick-start docs	Ничего

Порядок выполнения

Попросите Codex показать дерево только релевантных каталогов, серверную точку входа, схему conf.yaml и команды запуска.

```
rg --files -g README.md -g CLAUDE.md -g AGENTS.md -g pyproject.toml -g "**quick*"
rg -n "run_server|conf.yaml|tts_config|asr_config" .
```

Артефакты и доказательства

- [] Карта каталогов
- [] Список входных точек
- [] Список файлов, которые нельзя менять

Критерий завершения

- [] Вывод подтверждён файлами и строками
- [] Source tree неизменен

Эксперимент 4. Независимый аудит Codex

Проверить карту вторым запросом с другой формулировкой.

Атомарная цель. Проверить карту вторым запросом с другой формулировкой.

Score

Читать	Менять
Те же инструкции и входные точки	Ничего

Порядок выполнения

Начните новую задачу Codex и не подсказывайте вывод первого аудита. Запросите архитектурную карту и три главных риска запуска на Windows.

Сравните совпадения и расхождения; спорные утверждения перепроверьте командами.

Артефакты и доказательства

- ☐ Два независимых отчёта
- ☐ Таблица расхождений
- ☐ Команды проверки спорных мест

Критерий завершения

- ☐ Каждое принятое утверждение имеет локальное доказательство
- ☐ Изменений нет

Эксперимент 5. Аудит backend-контрактов

Определить допустимые LLM, ASR и TTS значения текущей версии.

Атомарная цель. Определить допустимые LLM, ASR и TTS значения текущей версии.

Score

Читать	Менять
Pydantic config models, example config, docs	Ничего

Порядок выполнения

```
rg -n "Literal.*tts|tts_model|asr_model|llm_provider|openai_compatible" src config* *.yaml
git status --short
```

Контроль. Не угадывайте строковые значения. Например, `pyttsx3_tts` должно буквально присутствовать в схеме текущей версии.

Артефакты и доказательства

- [] Допустимые enum/Literal значения
- [] Точные YAML paths
- [] Версия config schema

Критерий завершения

- [] Выбранные значения валидируются схемой
- [] Source tree неизменен

Эксперимент 6. Upstream baseline

Воспроизвести штатный запуск без изменения source и dependency versions.

Атомарная цель. Воспроизвести штатный запуск без изменения source и dependency versions.

Scope

Читать	Менять
README, quick-start, pyproject.toml, conf template	Только локальные generated files, если их создаёт штатный запуск

Порядок выполнения

```
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
uv sync
uv run run_server.py --verbose
```

Сохраните обезличенный лог запуска. Классифицируйте каждую ошибку: environment, configuration, dependency или source.

Контроль. Остановите сервер Ctrl+C после фиксации страницы и health/HTTP ответа.

Артефакты и доказательства

- [] Лог uv sync
- [] Verbose startup log
- [] Скриншот локальной страницы
- [] HTTP статус

Критерий завершения

- [] Baseline либо запущен, либо ошибка воспроизводима и классифицирована
- [] Версии зависимостей не редактировались

Эксперимент 7. Локальный Ollama VLM baseline

Подключить существующий ollama_llm только локальной конфигурацией.

Атомарная цель. Подключить существующий ollama_llm только локальной конфигурацией.

Score

Читать	Менять
conf.yaml и схема ollama_llm	Только untracked/local conf.yaml

Порядок выполнения

До правки покажите точные существующие YAML keys. После подтверждения задайте OpenAI-compatible endpoint Ollama.

```
# ожидаемые значения; реальные keys берите из текущей схемы
base_url: http://127.0.0.1:11434/v1
model: qwen3-v1:4b
```

```
ollama list
uv run run_server.py --verbose
```

Smoke tests: короткий text-only запрос и запрос со screenshot без персональных данных.

Артефакты и доказательства

- ☐ Запись endpoint/model
- ☐ Text-only ответ
- ☐ Ответ по screenshot
- ☐ Обезличенный лог

Критерий завершения

- ☐ Запросы не требуют cloud key
- ☐ Source/templates/dependencies неизменны

Эксперимент 8. Русский локальный voice baseline

Настроить sherpa_onnx_asr и pyttsx3_tts без cloud voice API.

Атомарная цель. Настроить sherpa_onnx_asr и pyttsx3_tts без cloud voice API.

Scope

Читать

Точные ASR/TTS keys и schema

Менять

Только local conf.yaml

Порядок выполнения

Получите список Windows voices PowerShell-совместимой командой:

```
@'
import pyttsx3
e = pyttsx3.init()
for v in e.getProperty("voices"):
    print(v.id, v.name, getattr(v, "languages", []))
'@ | uv run python -
```

В conf.yaml выберите существующие значения asr_model: sherpa_onnx_asr и tts_model: pyttsx3_tts. Для pyttsx3 выберите установленный русский voice id, не публикуя персональные пути.

Произнесите короткую фразу с уникальным числом, например: «Проверка локального голоса 4827». Затем воспроизведите короткий ответ.

Контроль. Динамики держите на умеренной громкости; после одного цикла остановите микрофон, чтобы исключить feedback loop.

Артефакты и доказательства

- [] Список pyttsx3 voices без персональных путей
- [] Запись выбранных ASR/TTS backends
- [] Transcript тестовой фразы с уникальным числом
- [] Подтверждение слышимого локального ответа

Критерий завершения

- [] ASR приемлемо распознал русскую фразу без cloud key
- [] TTS воспроизвёл ответ без внешнего voice API
- [] Нет бесконечного feedback loop
- [] Source tree неизменен

Эксперимент 8.5. Контрольная точка продолжения

Проверить эксперименты 1–8 и не повторять уже доказанное.

Атомарная цель. Проверить эксперименты 1–8 и не повторять уже доказанное.

Score

Читать	Менять
Артефакты экспериментов 1–8, git log/status	Ничего

Порядок выполнения

Создайте матрицу: выполнено / доказательство отсутствует / расходится с критерием.

- Если pytsx3 потребовал изменение source, отметьте эксперимент 8 как «работает, но критерий source unchanged не выполнен».
- Если evidence был committed в репо, не удаляйте историю автоматически; зафиксируйте отклонение.
- Если PR направлен в upstream вместо fork-only, не закрывайте его без отдельного решения.

```
git status --short
git log --oneline -10
git diff --stat
```

Артефакты и доказательства

- ☐ Матрица 1–8
- ☐ Список только недостающих доказательств
- ☐ Зафиксированные отклонения

Критерий завершения

- ☐ Нет слепого повторения
- ☐ Понятна безопасная стартовая точка эксперимента 9

Эксперимент 9. Минимальный runtime gateway

Создать отдельный loopback gateway с OpenAI-compatible chat endpoint.

Атомарная цель. Создать отдельный loopback gateway с OpenAI-compatible chat endpoint.

Scope

Читать	Менять
Документация FastAPI, OpenAI chat schema, Ollama/OpenRouter endpoints	Только новый sibling repo local-agent-gateway

Порядок выполнения

Попросите Codex сначала показать карту нового мини-проекта: pyproject.toml, app/main.py, app/settings.py, tests/.

```
Set-Location C:\dev\agent-lab
uv init local-agent-gateway
Set-Location .\local-agent-gateway
uv add fastapi "uvicorn[standard]" httpx pydantic-settings
uv add --dev pytest pytest-asyncio
```

Реализуйте GET /health и POST /v1/chat/completions. Bind — 127.0.0.1:8642. Все /v1 routes требуют Authorization: Bearer. Backend и model выбираются только из allowlist.

```
# .env.example — только имена, без реальных секретов
GATEWAY_TOKEN=replace-me
DEFAULT_BACKEND=ollama
OLLAMA_BASE_URL=http://127.0.0.1:11434/v1
```

Контроль. Не логируйте Authorization, prompt целиком и ответы пользователя.

Артефакты и доказательства

- [] Карта проекта
- [] OpenAPI schema
- [] Unit tests auth/health/allowlist
- [] Лог loopback bind

Критерий завершения

- [] Без token возвращается 401; запрещённая модель отклоняется
- [] Ollama отвечает через единый endpoint; секреты не tracked

Эксперимент 10. Интеграция gateway с VTuber

Перенаправить приложение на gateway только через local conf.yaml.

Атомарная цель. Перенаправить приложение на gateway только через local conf.yaml.

Score

Читать	Менять
LLM provider keys в conf.yaml	Только local conf.yaml

Порядок выполнения

```
# значения; точные имена keys подтвердите по текущей схеме
base_url: http://127.0.0.1:8642/v1
api_key: <локальный gateway token>
model: qwen3-v1:4b
```

Запустите gateway и Open-LLM-VTuber в двух PowerShell окнах. Сделайте text-only smoke test.

Контроль. Значение token не должно попасть в screenshot, лог или Git.

Артефакты и доказательства

- [] Запись URL/model без значения token
- [] Gateway request id
- [] Успешный ответ VTuber
- [] git status --short

Критерий завершения

- [] VTuber использует :8642
- [] Cloud backend не вызван
- [] Source tree VTuber неизменен

Эксперимент 11. Экран и голос через gateway

Проверить multimodal запрос и локальный voice loop одним контролируемым циклом.

Атомарная цель. Проверить multimodal запрос и локальный voice loop одним контролируемым циклом.

Score

Читать	Менять
Vision payload path, runtime logs	Только local config/evidence

Порядок выполнения

Используйте screenshot без личных данных. Сначала отправьте его в gateway с коротким вопросом, затем один раз произнесите фразу с уникальным числом.

Сопоставьте request id между VTuber и gateway; содержание речи в логах замените на [REDACTED], сохранив факт распознавания и число.

Контроль. После воспроизведения ответа выключите microphone capture и убедитесь, что новых циклов нет.

Артефакты и доказательства

- [] Обезличенный screenshot
- [] Gateway request id
- [] Transcript с уникальным числом
- [] Факт слышимого TTS

Критерий завершения

- [] Vision payload дошёл до локального VLM
- [] ASR/TTS остались локальными
- [] Нет feedback loop

Эксперимент 12. Совместимый adapter — только при доказанном разрыве

Добавить минимальный adapter лишь если фактическая схема VTuber несовместима с gateway.

Атомарная цель. Добавить минимальный adapter лишь если фактическая схема VTuber несовместима с gateway.

Score

Читать	Менять
Captured sanitized request/response schema, gateway tests	Только local-agent-gateway

Порядок выполнения

Сначала докажите разрыв минимальным failing test. Если формат уже совместим, зафиксируйте NOT REQUIRED и не пишите код.

Если adapter нужен, преобразуйте только несовместимые поля, сохраняя request id, status и нейтральные ошибки.

```
uv run pytest -q
```

Артефакты и доказательства

- [] Failing contract test или NOT REQUIRED
- [] Минимальный diff adapter
- [] Passing regression test

Критерий завершения

- [] Нет speculative code
- [] Совместимость доказана тестом
- [] VTuber source неизменен

Эксперимент 13. Инъекция отказов

Проверить безопасное поведение при недоступном backend, неверном token и timeout.

Атомарная цель. Проверить безопасное поведение при недоступном backend, неверном token и timeout.

Scope

Читать	Менять
Gateway error handling и tests	Только gateway tests/config

Порядок выполнения

- Неверный Bearer token → 401 без утечки ожидаемого token.
- Недоступный Ollama → 502/503 с нейтральным сообщением.
- Timeout → ограниченный ответ; процесс остаётся жив.
- Запрещённый backend/model → 400/403 до сетевого вызова.

```
uv run pytest -q
```

Артефакты и доказательства

- [] Таблица fault → HTTP status
- [] Тесты каждого отказа
- [] Обезличенные логи

Критерий завершения

- [] Нет stack trace и секретов в HTTP body
- [] Gateway восстанавливается после возврата backend

Эксперимент 14. Операционный runbook

Описать воспроизводимый запуск, остановку, диагностику и откат.

Атомарная цель. Описать воспроизводимый запуск, остановку, диагностику и откат.

Scope

Читать	Менять
Фактические команды и проверенные порты	Только local-agent-gateway/README.md

Порядок выполнения

Runbook должен включать порядок Ollama → gateway → VTuber, health checks, расположение локальной конфигурации, штатную остановку Ctrl+C и таблицу частых ошибок.

```
ollama serve
uv run uvicorn app.main:app --host 127.0.0.1 --port 8642
Invoke-WebRequest http://127.0.0.1:8642/health
uv run run_server.py --verbose
```

Контроль. Не вставляйте реальные tokens и персональные абсолютные пути.

Артефакты и доказательства

- [] README runbook
- [] Проверенные команды
- [] Таблица диагностики
- [] Rollback local config

Критерий завершения

- [] Новый пользователь может повторить baseline
- [] Каждая команда реально проверена

Эксперимент 15. Поставка в fork

Закрепить gateway и отчёт отдельным reviewable commit/PR без изменения upstream main.

Атомарная цель. Закрепить gateway и отчёт отдельным reviewable commit/PR без изменения upstream main.

Scope

Читать

Diff, tests, branch, remotes

Менять

Только gateway репо и согласованные evidence

Порядок выполнения

```
git status --short
git diff --check
uv run pytest -q
git diff --stat
```

После проверки создайте осмысленный commit, push рабочей ветки и draft PR в собственный fork. Не включайте .env, audio, персональные screenshot и большие model files.

Контроль. Любая публикация — отдельное подтверждаемое действие. Перед push покажите точный scope.

Артефакты и доказательства

- [] Краткий diff
- [] Полный реальный test output
- [] git status --short
- [] Ссылка на draft PR в fork

Критерий завершения

- [] CI/локальные тесты зелёные
- [] Секретов и персональных данных нет
- [] main и upstream не изменены

Definition of Done

- [] Codex использовался как инженерный агент с plan-before-write и атомарным scope.
- [] Open-LLM-VTuber запускается на воспроизводимом upstream baseline.
- [] Ollama обеспечивает локальный text/vision baseline.
- [] sherpa_onnx_asr и pyttsx3_tts дают один русский voice cycle без cloud key и feedback loop.
- [] Gateway слушает только 127.0.0.1, требует Bearer auth и ограничивает backend/model.
- [] VTuber использует gateway через локальный conf.yaml без source/template/dependency edits.
- [] Ошибки auth/backend/timeout проверены тестами и не раскрывают секреты.
- [] Runbook соответствует фактически проверенным командам.
- [] Поставка выполнена в рабочую ветку собственного fork; upstream main не изменён.

Карта типовых ошибок

Симптом	Класс	Первое действие
Pydantic literal_error для tts_model	Configuration	Сверить значение со схемой текущей версии
Объект TTS не имеет ожидаемого метода	Source/backend contract	Воспроизвести минимально; не патчить source без нового scope
Connection refused :11434	Environment	Проверить Ollama и loopback port
401 от :8642	Configuration/security	Проверить наличие Bearer без печати значения
502/503 gateway	Backend	Проверить /health и доступность выбранного backend
PowerShell пугается на <<'PY'	Shell syntax	Использовать @'...'@ uv run python -

Шаблон итогового отчёта

Заполняйте ссылками на обезличенные артефакты. Не подменяйте отсутствующий тест описанием.

Раздел	Содержание
Среда	Версии Windows, Git, uv, Ollama; без имени пользователя
Baseline	Commit/tag upstream и результат запуска
Модели	Ollama model; cloud backend — только если включался
Voice	ASR/TTS backend, voice name, transcript с уникальным числом
Gateway	Bind, auth, allowlist, health, request id
Тесты	Команды, полный exit code и краткий вывод
Отклонения	Что не выполнено и почему
Git	Branch, commit, PR в fork, clean/known status

Промпт для финального review

Проведи read-only review итогов практикума.
 Проверь доказательства против Definition of Done.
 Для каждого пункта верни PASS / FAIL / NOT PROVEN.
 Не считай словесное утверждение тестом.
 Не читай секреты и персональные данные.
 Отдельно проверь: source tree VTuber, gateway auth/allowlist,
 локальность ASR/TTS, отсутствие feedback loop и target PR.

Справочные ссылки

Codex documentation: <https://developers.openai.com/codex/>

OpenAI API reference: <https://platform.openai.com/docs/api-reference>

Open-LLM-VTuber: <https://github.com/Open-LLM-VTuber/Open-LLM-VTuber>

Ollama API: <https://docs.ollama.com/api>

OpenRouter API: <https://openrouter.ai/docs/api-reference/overview>

FastAPI: <https://fastapi.tiangolo.com/>

uv: <https://docs.astral.sh/uv/>

GitHub CLI: <https://cli.github.com/manual/>

Примечание. Интерфейсы и версии меняются. Перед выполнением сверяйте команды с документацией и схемой именно установленной версии.